



A rede que conecta talentos iniciantes em tecnologia a recrutadores

Integrantes do grupo:

Bianca Ricci Lima — RA: 082230019

Gustavo Sgrignoli Marmo — RA: 082230028

Professores(as):

Israel Florentino

Leide Aparecida

São Bernardo do Campo — SP

2026

Sumário

Seção 1 — Visão e Escopo do Projeto

- 1.1 Documento de Visão
- 1.2 Requisitos Funcionais e Não Funcionais
- 1.3 Priorização Estratégica do Backlog
- 1.4 Definição do MVP e Roadmap de Releases

Seção 2 — Modelagem de Software (UML)

- 2.1 Diagrama de Classes
- 2.2 Diagrama de Casos de Uso
- 2.3 Diagramas de Sequência

Seção 3 — Modelagem de Processos (BPMN)

- 3.1 Diagrama BPMN — Processo de Candidatura e Conexão

Seção 4 — Especificação de Funcionalidade para Desenvolvimento

- 4.1 História de Usuário Seleccionada
- 4.2 Wireframes e Storyboard
- 4.3 Casos de Teste Funcionais

Apêndice — Link do Repositório do Projeto

Seção 1 — Visão e Escopo do Projeto

1.1 Documento de Visão

Problema

O ingresso no mercado de trabalho em tecnologia é particularmente difícil para estudantes, recém-formados e profissionais em transição de carreira. Plataformas generalistas como o LinkedIn são saturadas, possuem alta concorrência de candidatos seniores e não exibem com clareza o que diferencia um talento iniciante: seus projetos práticos, suas tecnologias em estudo e seu momento de carreira. Por outro lado, recrutadores e empresas que buscam contratar estagiários e desenvolvedores juniores enfrentam dificuldade em encontrar candidatos com perfis adequados, ficando soterrados em currículos genéricos que não evidenciam o repertório técnico real do candidato.

Solução proposta

O Firstech é uma rede social profissional especializada em conectar talentos iniciantes em tecnologia (estudantes, estagiários e devs juniores) a recrutadores em busca desses perfis. A plataforma oferece um portfólio rico orientado a projetos, um feed social com posts técnicos, uma busca unificada de pessoas, vagas e publicações, sistema de conexões e mensagens diretas — tudo em um ambiente focado em tecnologia, sem o ruído de redes generalistas.

Público-alvo

- Candidatos: estudantes de cursos de TI (Ciência da Computação, ADS, Engenharia de Software, etc.), estagiários, desenvolvedores juniores e pessoas em transição de carreira que querem dar os primeiros passos na área tech.
- Recrutadores: tech recruiters, head-hunters e profissionais de RH especializados em tecnologia, em empresas de qualquer porte (de startups a grandes corporações), que precisam preencher posições de estágio, júnior e trainee.

Objetivos de negócio

- Aumentar a taxa de matching entre vagas de entrada e candidatos qualificados, medida pela proporção de candidaturas que avançam para entrevista.
- Reduzir o tempo médio (em dias) entre a publicação de uma vaga e o preenchimento por meio da plataforma.
- Elevar o engajamento da comunidade técnica iniciante por meio de posts, projetos publicados e conexões realizadas (DAU/MAU, posts por usuário/mês, conexões aceitas).
- Conquistar, em até 12 meses após o lançamento, ao menos 500 candidatos ativos e 30 recrutadores recorrentes na plataforma.

1.2 Requisitos Funcionais e Não Funcionais

Os requisitos abaixo refletem o escopo total identificado para o produto. A definição do MVP (Seção 1.4) seleciona um subconjunto destes para a primeira versão entregável.

Requisitos Funcionais

ID	Descrição
RF01	O sistema deve permitir o cadastro de usuários com papéis distintos: CANDIDATO e RECRUTADOR.
RF02	O sistema deve oferecer fluxo de onboarding multi-step com coleta progressiva de dados (Progressive Profiling).
RF03	O sistema deve autenticar usuários via e-mail e senha, retornando um token JWT.
RF04	O sistema deve permitir a recuperação de senha por meio de link enviado por e-mail.
RF05	O sistema deve permitir que candidatos montem um portfólio com Sobre, Experiências, Habilidades, Certificações e Projetos (incluindo upload de imagem e link do GitHub).
RF06	O sistema deve permitir que candidatos enviem foto de perfil (avatar) e imagem de banner.
RF07	O sistema deve permitir que recrutadores publiquem, editem, encerrem e excluam vagas com título, descrição, localidade, modalidade, nível, salário e tags.
RF08	O sistema deve listar vagas ativas para todos os usuários, com filtros por modalidade (remoto, híbrido, presencial) e nível (estágio, júnior, pleno).
RF09	O sistema deve permitir que usuários publiquem posts no feed com título, conteúdo, tags e (no caso de recrutadores) vinculação opcional a uma vaga ativa.
RF10	O sistema deve permitir curtir e comentar posts.
RF11	O sistema deve oferecer busca unificada por pessoas, vagas e publicações.
RF12	O sistema deve permitir que usuários enviem, aceitem e recusem pedidos de conexão.
RF13	O sistema deve permitir que usuários conectados troquem mensagens diretas em tempo quase real.
RF14	O sistema deve exibir o perfil público de qualquer usuário, incluindo status de conexão relativo ao visualizador.
RF15	O sistema deve exibir notificações de mensagens não lidas e de pedidos de conexão pendentes.

Requisitos Não Funcionais

ID	Categoria	Descrição
RNF01	Segurança	Senhas devem ser armazenadas com BCrypt (strength 12) e a autenticação deve usar JWT stateless com refresh token rotativo.
RNF02	Segurança	Endpoints sensíveis devem exigir autorização baseada em papel (RECRUTADOR, CANDIDATO, ADMINISTRADOR).
RNF03	Usabilidade	A interface deve ser responsiva, com tema dark mode coerente, acessível em desktop e dispositivos móveis.
RNF04	Desempenho	As páginas principais (feed, vagas, perfil) devem carregar em menos de 2 segundos em condições normais.
RNF05	Confiabilidade	Falhas de validação ou de regra de negócio devem retornar mensagens claras em português, sem expor stack trace.
RNF06	Manutenibilidade	O backend deve seguir a arquitetura em camadas (controller, service, repository) com DTOs separados das entidades.
RNF07	Portabilidade	O sistema deve rodar em Java 17 com Spring Boot, podendo ser empacotado em container Docker.
RNF08	Privacidade	Apenas o próprio usuário pode editar seu portfólio; conexões são bidirecionais e podem ser desfeitas a qualquer momento.

1.3 Priorização Estratégica do Backlog

Para definir o escopo do MVP, a equipe aplicou duas técnicas complementares de priorização: MoSCoW e Matriz de Impacto × Esforço.

Justificativa da escolha das técnicas

- **MoSCoW:** escolhida por sua clareza para comunicar prioridades a todo o grupo (técnico e não técnico) em ciclos curtos de planejamento. A categorização em Must, Should, Could e Won't é ideal para definir rapidamente o que entra no MVP e o que fica para releases seguintes. Como o Firstech é um produto novo, sem histórico de uso, MoSCoW evita análises quantitativas precoces que dependeriam de dados ainda inexistentes.
- **Matriz Impacto × Esforço:** escolhida por trazer uma dimensão econômica ao MoSCoW. Como o grupo é pequeno e o tempo de desenvolvimento é limitado, é essencial priorizar funcionalidades com alto valor percebido e custo de implementação proporcional. A matriz ajuda a identificar quick wins (alto impacto, baixo esforço) e a evitar armadilhas (baixo impacto, alto esforço), refinando o resultado do MoSCoW.

Aplicação da técnica MoSCoW

Categoria	Funcionalidade	RF relacionado
MUST HAVE		
Must	Cadastro e autenticação de usuários (CANDIDATO/RECRUTADOR) com JWT	RF01, RF03
Must	Onboarding multi-step (Progressive Profiling)	RF02
Must	Portfólio do candidato (Sobre, Experiências, Skills, Projetos)	RF05, RF06
Must	Publicação e listagem de vagas por recrutadores	RF07, RF08
Must	Feed com posts (criar, listar, curtir, comentar)	RF09, RF10
Must	Perfil público com status de conexão	RF14
SHOULD HAVE		
Should	Sistema de conexões (enviar, aceitar, recusar pedidos)	RF12
Should	Mensagens diretas entre conexões	RF13
Should	Busca unificada (pessoas, vagas, posts)	RF11
Should	Recuperação de senha por e-mail	RF04
COULD HAVE		
Could	Notificações em tempo real de mensagens e conexões	RF15
Could	Recomendação algorítmica de vagas por afinidade de skills	—
Could	Filtros avançados na busca (faixa salarial, localidade, etc.)	RF08
WON'T HAVE (esta release)		
Won't	Aplicativo móvel nativo (iOS/Android)	—
Won't	Integração com LinkedIn e GitHub para importar perfil	—
Won't	Verificação de telefone via SMS	—
Won't	Vídeo-currículo e entrevista por vídeo na plataforma	—

Aplicação da Matriz Impacto × Esforço

Cada funcionalidade foi avaliada em duas dimensões, com pontuação de 1 (baixo) a 5 (alto). O resultado refina a definição do MVP, validando os itens Must do MoSCoW e identificando candidatos a antecipar/postergar.

Funcionalidade	Impacto (1-5)	Esforço (1-5)	Classificação
Cadastro + autenticação JWT	5	3	Big Bet (essencial)
Portfólio do candidato	5	4	Big Bet (essencial)
Publicação de vagas (recrutador)	5	2	Quick Win
Feed de posts (criar/listar)	4	2	Quick Win
Curtir e comentar posts	3	1	Quick Win
Busca unificada	4	3	Big Bet
Sistema de conexões	4	3	Big Bet
Mensagens diretas	4	4	Big Bet
Notificações em tempo real (WS)	3	5	Money Pit (postergar)
App mobile nativo	4	5	Money Pit (postergar)
Recomendação algorítmica de vagas	3	4	Fill-in (avaliar depois)

Conclusão da análise: a combinação MoSCoW + Impacto × Esforço confirma que os itens Must do MoSCoW concentram os Big Bets essenciais e os Quick Wins, que entregam valor imediato ao usuário com esforço proporcional. Mensagens diretas e conexões, apesar de Big Bets, entram no MVP por serem o que torna a rede social efetivamente uma rede. Notificações em tempo real e app mobile foram classificados como Money Pit e ficaram fora do MVP, sendo realocados para releases futuras.

1.4 Definição do MVP e Roadmap de Releases

MVP — Release 1.0

O MVP entrega o ciclo mínimo essencial da rede social: o candidato consegue se cadastrar, montar seu portfólio e ser encontrado; o recrutador consegue publicar vagas e descobrir candidatos. A interação social (posts, conexões, mensagens) está presente em sua forma básica, sustentando a proposta de valor de rede.

- Cadastro e autenticação (JWT, BCrypt).
- Onboarding multi-step para candidatos e recrutadores.
- Portfólio do candidato (perfil, Sobre, Experiências, Habilidades, Certificações, Projetos, avatar e banner).

- Publicação e gerenciamento de vagas (recrutador).
- Listagem pública de vagas com filtros básicos.
- Feed de posts: criar, listar, curtir, comentar.
- Perfil público de usuário.
- Sistema de conexões (enviar, aceitar, recusar).
- Mensagens diretas entre conexões.
- Busca unificada (pessoas, vagas, posts).

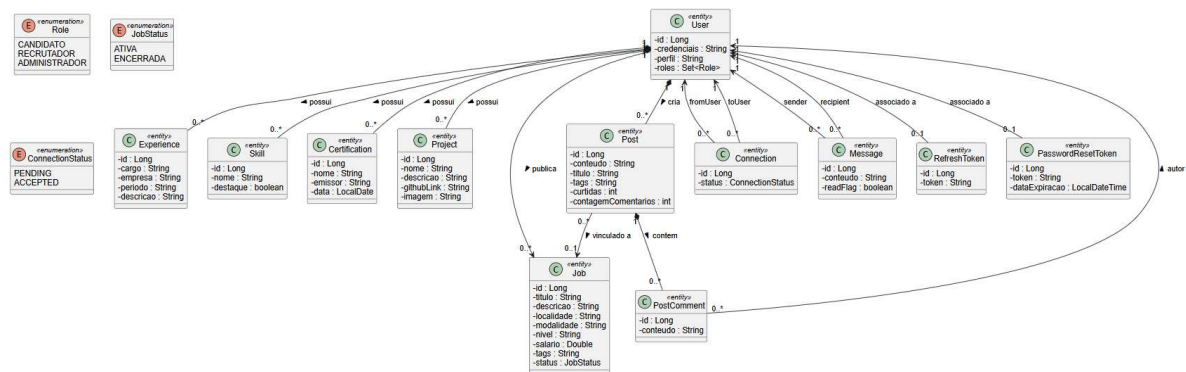
Roadmap de Releases

Release	Tema	Funcionalidades principais
1.0 (MVP)	Validação da proposta	Cadastro, portfólio, vagas, feed, conexões, mensagens, busca.
1.1	Onboarding e ativação	Recuperação de senha por e-mail, verificação de e-mail, filtros avançados de vagas, sugestões de conexão.
1.2	Engajamento social	Notificações em tempo real (WebSocket) para mensagens e pedidos de conexão; reações em posts; tags trending.
1.3	Descoberta inteligente	Recomendação de vagas por afinidade de skills do portfólio; feed personalizado por tags seguidas.
2.0	Mobile e integrações	Aplicativo móvel nativo (iOS/Android), integração com LinkedIn/GitHub para importar perfil, integração com calendário para entrevistas.
2.1	Recrutamento ativo	Painel analítico para recrutador, candidaturas com tracking de status (in review, entrevista, oferta), templates de mensagem.

Seção 2 — Modelagem de Software (UML)

2.1 Diagrama de Classes

O Diagrama de Classes representa a estrutura estática do domínio do Firsttech, refletindo as entidades JPA implementadas no backend Spring Boot. Foram modeladas as classes principais que compõem o MVP, com seus atributos, principais métodos e relacionamentos.

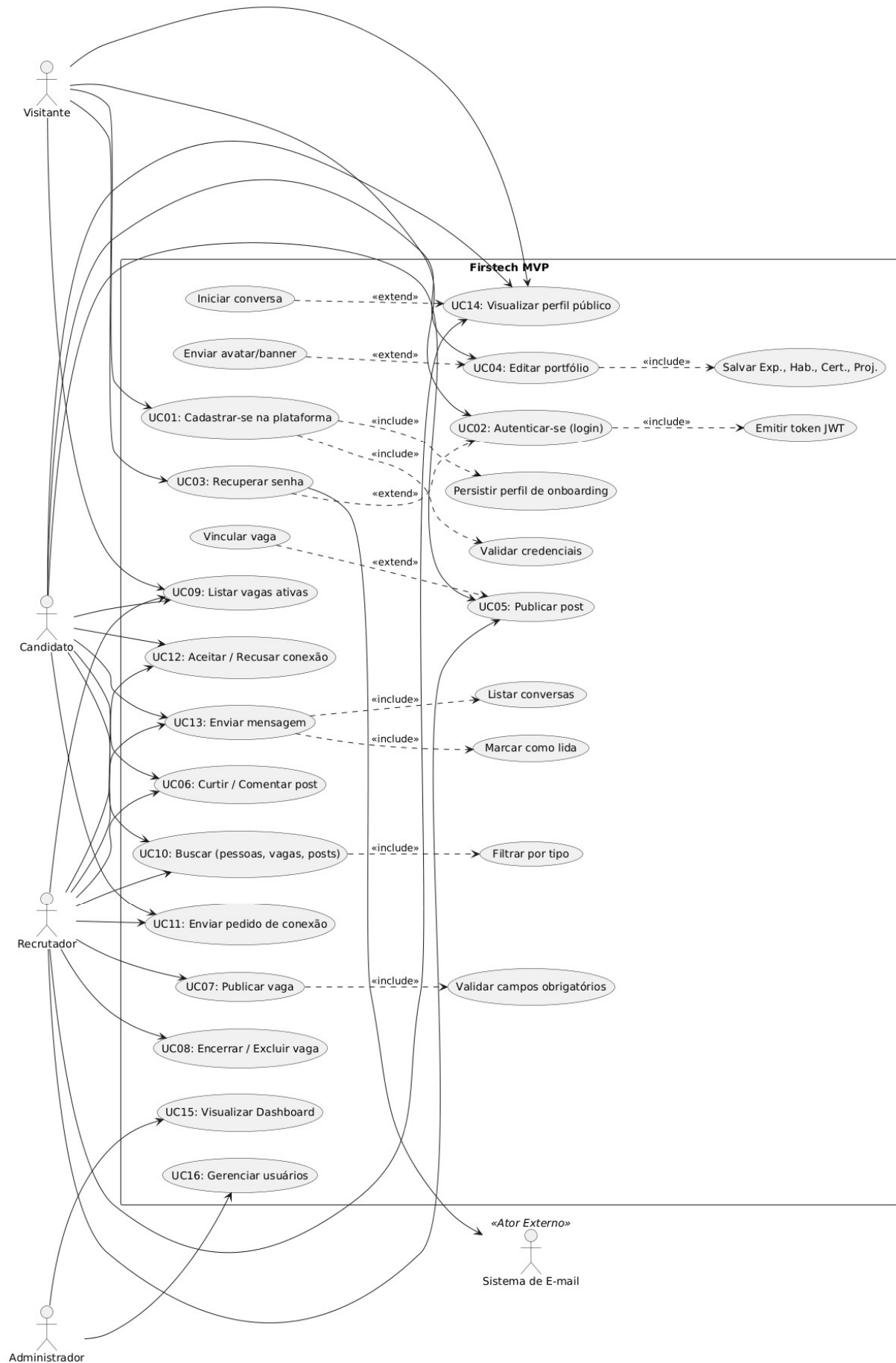


Observações de notação

- Os atributos seguem a visibilidade padrão (- private) com getters/setters gerados por Lombok.
- As entidades JPA usam Long como tipo de ID e @GeneratedValue(IDENTITY).
- As enumerações Role, JobStatus e ConnectionStatus são representadas como classes estereotipadas <<enumeration>>.
- As composições (losango cheio) indicam que filhos não fazem sentido sem o pai (Experience sem User, por exemplo).

2.2 Diagrama de Casos de Uso

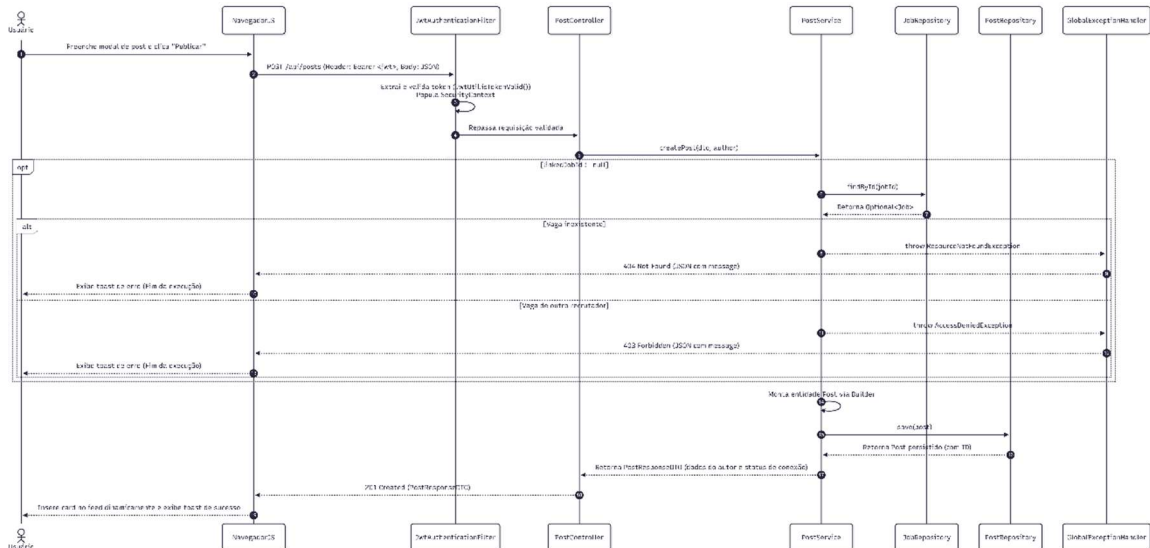
O Diagrama de Casos de Uso identifica os atores que interagem com o Firsttech e as funcionalidades do MVP. Relacionamentos <<include>> e <<extend>> são utilizados para evitar duplicação e modelar fluxos opcionais.



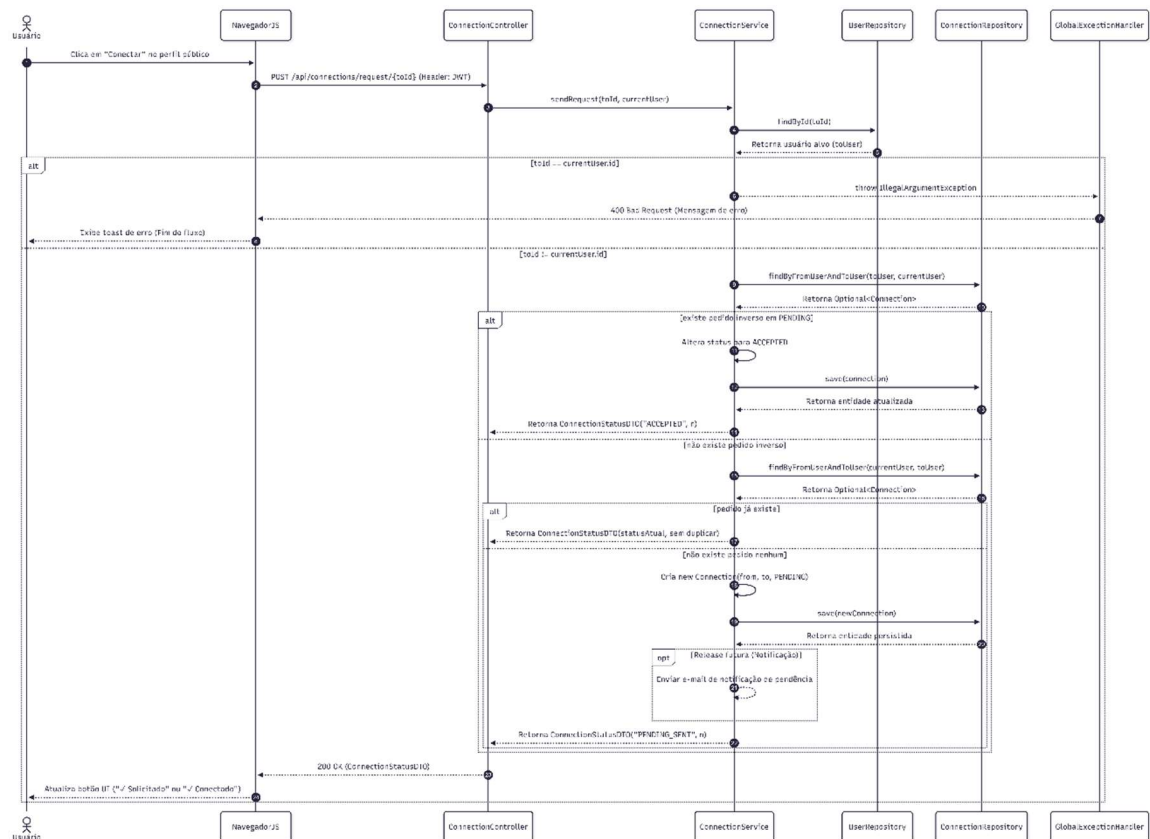
2.3 Diagramas de Sequência

Foram modelados dois casos de uso críticos para o valor do produto: a publicação de um post (UC05) e o envio de pedido de conexão (UC11). Cada diagrama detalha a troca de mensagens entre objetos, com lifelines, mensagens síncronas, retornos e fragmentos alt/loop quando necessário.

2.3.1 Diagrama de Sequência — Publicar Post (UC05)



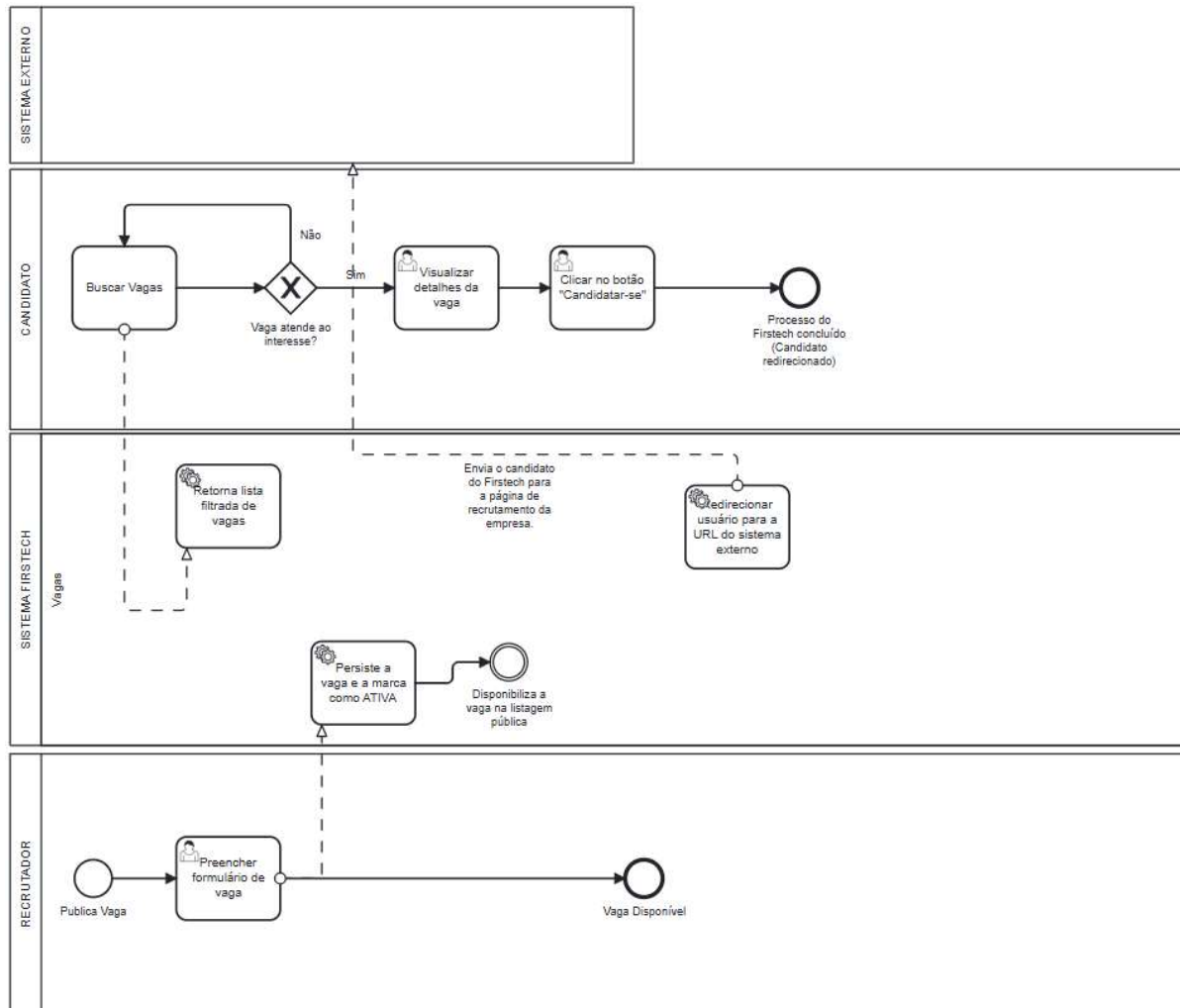
2.3.2 Diagrama de Sequência — Enviar Pedido de Conexão (UC11)



Seção 3 — Modelagem de Processos (BPMN)

3.1 Diagrama BPMN — Processo de Candidatura à Vaga

O processo central modelado é o ciclo de descoberta-candidatura no Firstech: como um candidato encontra uma vaga compatível e é redirecionado para o site externo de recrutamento da empresa contratante. Este processo conecta os atores do sistema e atravessa todas as funcionalidades centrais do MVP.



Seção 4 — Especificação de Funcionalidade para Desenvolvimento

4.1 História de Usuário Selecionada

Foi selecionada uma história de usuário crítica, interativa e central para a proposta de valor do Firstech: a publicação de uma nova vaga por um recrutador. Esta funcionalidade é o motor da rede social — sem vagas publicáveis pelo recrutador, candidatos não têm oportunidades para descobrir, e a plataforma perde sua razão de existir.

HISTÓRIA DE USUÁRIO — HU07	
Título	Publicar uma nova vaga
Persona	Recrutador
Narrativa	Como recrutador autenticado, quero publicar uma nova vaga preenchendo título, empresa contratante, localidade, modalidade, nível, salário, descrição e tags, para que candidatos compatíveis encontrem a oportunidade na listagem de vagas e possam se candidatar.
Prioridade	Must Have (MVP)
CrITÉrios de aceite	• Apenas usuários com papel RECRUTADOR conseguem acessar a função. • O campo Título é obrigatório; demais campos são opcionais (mas recomendados). • Ao salvar, a vaga é gravada com status ATIVA e a data de hoje em createdAt. • A vaga aparece imediatamente na lista de "Minhas vagas" do recrutador e na listagem pública para candidatos. • Toast de confirmação "Vaga publicada com sucesso!" é exibido.

4.2 Wireframes e Storyboard

O storyboard abaixo descreve o fluxo de cinco telas que o recrutador percorre para publicar uma nova vaga. Os wireframes correspondentes (a serem desenhados em ferramenta como Figma, Excalidraw ou Balsamiq) devem detalhar cada campo, botão e mensagem.

Storyboard — Fluxo de telas

```

+-----+
| [=] Firstech      Home* | Jobs | Rede | Portfolio  [Avatar] [ + Nova Vaga ] |
+-----+
| +-----+ +-----+ +-----+ |
| | [Avatar] | | Escreva um post... | | Suas vagas ativas | |
| | Nome do Recrutador | | [ Publicar ] | | | |
| | Tech Recruiter | +-----+ | [!] Nenhuma vaga | |
| | | | [Avatar] Usuário A | | ativa no momento. | |
| | Conexões: 1.204 | | Acabamos de lançar a nova | |
| | Visualizações: 142 | | funcionalidade de IA! | |
| +-----+ | [Curtir] [Comentar] | | [+ Publicar vaga ] | |
| | | +-----+ +-----+ |
| | | |
| | | (scroll infinito feed...) |
+-----+
+=====+
| // Fundo escuro semi-transparente (Overlay) // |
+-----+
| | [Mala] Nova vaga - Firstech [X] | |
+-----+
| | Título da vaga* | | |
| | [ Título da vaga (foco capturado aqui) ] | |
| | Empresa contratante | |
| | [ Buscar empresa (opcional)... ] | |
| | Localidade Modalidade | |
| | [ Buscar cidade (IBGE)... ] [ Selecione... v ] | |
| | Nível Salário | |
| | [ Selecione... v ] [ Ex: R$ 3.000 - 5.000 ] | |
| | Descrição da vaga | |
| | [ Escreva os requisitos, responsabilidades e benefícios... ] | |
| | [ ] | |
| | [ ] | |
| | Tags (habilidades, tecnologias) | |
| | [ Digite a tag e aperte Enter ou vírgula... ] | |
+-----+
| | A vaga ficará visível para todos os candidatos | |
| | [ Cancelar ] [ > ] Publicar vaga (Roxo) ] | |
+-----+
+=====+

```



```

=====
| // Fundo escuro semi-transparente (Overlay) //
|
| +-----+
| | [Mala] Nova vaga - Firsttech | [X] |
| +-----+
| | Título da vaga* | |
| | [ Desenvolvedor Java Júnior ] |
| |
| | Empresa contratante |
| | [ [Logo] Firstech ] |
| |
| | Localidade | Modalidade |
| | [ São Bernardo do Campo, SP ] [ Híbrido v ] |
| |
| | Nível | Salário |
| | [ Júnior v ] [ R$ 4.000 - 6.000 ] |
| |
| | Descrição da vaga |
| | [ Buscamos um desenvolvedor com foco em backend para nossos ] |
| | [ sistemas core. Experiência com arquitetura MVC é um diferencial.] |
| | [ ] |
| |
| | Tags (habilidades, tecnologias) |
| | [ [Java (x)] [Spring Boot (x)] [MySQL (x)] ] |
| +-----+
| | A vaga ficará visível para todos os candidatos |
| | [ Cancelar ] [ (*) Aguarde... (Desab) ] |
| +-----+
|
=====

+-----+
| [=] Firsttech | Home* | Jobs | Rede | Portfolio | [Avatar] | [ + Nova Vaga ] |
+-----+
|
| +-----+ +-----+ +-----+
| | [Avatar] | | Escreva um post... | | Suas vagas ativas |
| | Nome do Recrutador | | [ Publicar ] | | |
| | Tech Recruiter | +-----+ | +-----+
| | | | [Avatar] Usuário A | | [L] Desenvolved. |
| | Conexões: 1.204 | | Acabamos de lançar a nova | | [ATIVA] Java Jr |
| | Visualizações: 142 | | funcionalidade de IA! | | SBC, SP Híbrido |
| +-----+ | [Curtir] [Comentar] | | [Java] [Spring] |
| | | +-----+ | | Publicada agora |
| | | | | | [Edi][Enc][Exc] |
| | | | +-----+
| | | | [+ Publicar vaga] |
| | | +-----+
|
| [v] Vaga "Desenvolvedor Java Júnior" publicada com sucesso! [x]
|
+-----+

```

```

+-----+
| [=] Firstech      Home | Jobs* | Rede | Portfolio  [Avatar] [ Meu Perfil] |
+-----+
| FILTROS           [ Buscar por título, empresa ou tecnologia... (Q) ] |
|
| Modalidade        +-----+
| [ ] Remoto        | [Logo Firstech] Desenvolvedor Java Júnior |
| [x] Híbrido        | Firstech · São Bernardo do Campo, SP · Híbrido |
| [ ] Presencial    | Buscamos um desenvolvedor com foco em backend para |
|                   | nossos sistemas core. Experiência com arquitetura.. |
| Nível             | R$ 4.000 – 6.000 |
| [ ] Estágio        | Tags: [Java] [Spring Boot] [MySQL] |
| [x] Júnior         |                                     [ Candidatar ] |
| [ ] Pleno          +-----+
| [ ] Sênior         | [Logo XYZ]      Engenheiro de Dados Pleno |
|                   | Empresa XYZ · Remoto · Pleno |
+-----+

```

4.3 Casos de Teste Funcionais

Foram especificados três casos de teste para a história HU07: um caminho feliz (publicação bem-sucedida) e dois caminhos de erro (validação no front-end e autorização no back-end).

CT01 — Publicação bem-sucedida de vaga (caminho feliz)

CT01 — Publicação bem-sucedida de vaga	
Objetivo	Verificar que um recrutador autenticado consegue publicar uma vaga com todos os campos preenchidos e que a vaga é persistida no banco com status ATIVA, aparecendo imediatamente na listagem.
Pré-condições	• Usuário cadastrado com papel RECRUTADOR e autenticado (token JWT válido em localStorage). • Aplicação rodando em http://localhost:8081. • Banco de dados acessível (H2 em memória).
Dados de entrada	• Título: "Desenvolvedor Java Júnior" • Empresa: "Firstech" • Localidade: "São Bernardo do Campo, SP" • Modalidade: "Híbrido" • Nível: "Júnior" • Salário: "R\$ 4.000 – 6.000" • Descrição: "Buscamos dev Java apaixonado por tecnologia para integrar nosso time de backend." • Tags: ["Java", "Spring Boot", "MySQL"]
Passos	1. Estar na tela Dashboard logado como recrutador.

	2. Clicar no botão "+ Nova Vaga" no canto superior direito. 3. Verificar que o modal abre com o foco no campo Título. 4. Preencher cada campo com os dados de entrada listados acima. 5. Adicionar as três tags pressionando Enter após cada uma. 6. Clicar no botão "Publicar vaga". 7. Aguardar o estado de loading do botão.
Resultado esperado	• O modal fecha automaticamente após sucesso (aprox. 1 s). • Toast verde "Vaga "Desenvolvedor Java Júnior" publicada com sucesso!" aparece no canto inferior direito. • Um novo card de vaga é injetado no topo da lista "Minhas vagas" sem reload. • Endpoint POST /api/jobs retornou HTTP 201 Created. • Resposta JSON inclui id, title, company, status "ATIVA" e createdAt com a data de hoje. • A vaga aparece na listagem pública /api/jobs (consultável também sem autenticação).
Critério de aceite	Todos os resultados esperados confirmados; nenhum erro no console; nenhuma requisição falhou.

CT02 — Tentativa de publicação com título vazio (caminho de erro — validação)

CT02 — Validação de campo obrigatório (título vazio)	
Objetivo	Verificar que a tentativa de publicar uma vaga sem preencher o título obrigatório é bloqueada antes da chamada ao backend, com feedback visual claro ao usuário.
Pré-condições	• Usuário cadastrado com papel RECRUTADOR e autenticado. • Modal de "Nova vaga" aberto.
Dados de entrada	• Título: (vazio) • Demais campos: indiferentes (podem estar preenchidos ou vazios)
Passos	1. Abrir o modal de Nova Vaga. 2. Deixar o campo Título completamente vazio. 3. Preencher opcionalmente outros campos (ex: descrição "Teste"). 4. Clicar no botão "Publicar vaga".
Resultado esperado	• Nenhuma requisição HTTP é enviada ao backend (verificável no DevTools Network). • O foco é movido automaticamente para o campo Título. • A borda do campo Título fica vermelha por aproximadamente 2 segundos.

	• O modal permanece aberto. • Nenhuma vaga é criada no banco de dados. • O botão Publicar vaga não fica em estado loading.
Critério de aceite	Validação client-side bloqueia a submissão e o usuário recebe feedback visual imediato; nenhum dado inválido é persistido.

CT03 — Tentativa de publicação por usuário sem papel de recrutador (caminho de erro — autorização)

CT03 — Bloqueio por falta de autorização (papel CANDIDATO)	
Objetivo	Verificar que um usuário autenticado com papel CANDIDATO (não RECRUTADOR) é impedido pelo backend de publicar uma vaga, mesmo se conseguir burlar o front-end.
Pré-condições	• Usuário cadastrado com papel CANDIDATO (e apenas CANDIDATO) e autenticado. • Token JWT do candidato disponível. • Aplicação rodando normalmente.
Dados de entrada	• POST /api/jobs com header Authorization: Bearer <token_candidato> • Body JSON válido: {"title":"Tentativa indevida", "description":"...", "location":"São Paulo", ...}
Passos	1. Obter um token JWT válido fazendo login como CANDIDATO. 2. Por meio de uma ferramenta como Postman, Insomnia ou cURL, enviar a requisição POST /api/jobs com o token do candidato no header Authorization. 3. Observar o status HTTP da resposta e o corpo retornado. 4. Consultar GET /api/jobs como recrutador para confirmar que a vaga não foi criada.
Resultado esperado	• Resposta HTTP 403 Forbidden. • Corpo JSON contém campos timestamp, status: 403, error: "Forbidden" e mensagem descritiva. • Nenhum registro é gravado na tabela jobs. • A anotação @PreAuthorize("hasAuthority('RECRUTADOR')") no JobController.createJob() bloqueou a requisição antes que o método fosse executado.
Critério de aceite	Backend rejeita a requisição com 403 e nenhuma vaga é criada, comprovando que a segurança por papel está aplicada no servidor (e não apenas escondida no front-end).

Link do Repositório do Projeto

Repositório do código (Linguagem de Programação II)

<https://github.com/gustavomarmo/firstech>

Stack técnica resumida

- Linguagem: Java 17.
- Framework: Spring Boot 3.2.5 (Web, WebFlux, Data JPA, Security, Validation, Thymeleaf, Mail).
- Persistência: Spring Data JPA com H2 (em memória) para desenvolvimento.
- Segurança: Spring Security + JWT (jjwt 0.12.3) + BCrypt (strength 12).
- Documentação de API: SpringDoc OpenAPI 2.5 (Swagger UI).
- Front-end: Thymeleaf (server-side rendering) + Tailwind CSS (CDN) + JavaScript vanilla.
- Build: Maven (Spring Boot Maven Plugin).
- Testes: JUnit 5 + Mockito + Spring Security Test.

Como executar localmente

- Pré-requisitos: JDK 17 e Maven instalados (ou usar o Maven Wrapper).
- Clonar o repositório.
- Na raiz do projeto, executar: `./mvnw spring-boot:run` (Linux/macOS) ou `mvnw.cmd spring-boot:run` (Windows).
- Acessar a aplicação em `http://localhost:8081`.
- Usuários pré-cadastrados (seed em DataInitializer): `admin@example.com / Admin@123` (recrutador-admin), `candidato@example.com / Admin@123` (candidato júnior).
- Console H2 disponível em `http://localhost:8081/h2-console` (JDBC URL: `jdbc:h2:mem:authdb`, usuário: `sa`, senha: `1234`).
- Documentação Swagger disponível em `http://localhost:8081/swagger-ui.html`.